

CSMA/CA and the IEEE 802.11 MAC in INET

A code-level walkthrough of how INET implements contention-based channel access — from the generic `CsmaCaMac` up through DCF, EDCA/HCF, and the (unimplemented) PCF/HCCA — with exact classes, parameters, and formulas, and a map of where to make changes.

Contents

1	Two MAC implementations, one idea	2
2	Generic CSMA/CA: <code>CsmaCaMac</code>	2
2.1	Parameters (<code>CsmaCaMac.ned</code>)	2
2.2	The state machine	3
2.3	Backoff and contention window growth	4
3	802.11 MAC architecture overview	4
3.1	Which class does what	5
4	DCF (Distributed Coordination Function)	5
4.1	Timing parameters	6
4.2	The Contention module: backoff + IFS as one FSM	6
4.3	Frame sequences: a small combinator grammar	7
4.4	RTS/CTS, NAV and the duration field	8
4.5	Retry, recovery, and contention-window growth	8
4.6	A complete DCF transmission, end to end	9
5	HCF / EDCA (Enhanced Distributed Channel Access)	9
5.1	Four independent contenders, one radio	10
5.2	Per-AC timing: AIFS instead of DIFS	10
5.3	Internal (virtual) collisions	11
5.4	TXOP bursting	11
5.5	Block acknowledgement (optional)	11
6	PCF and HCCA — status: not implemented	12
6.1	What exists today	12
6.2	What PCF/HCCA would need to do, conceptually	13
6.3	A concrete starting point, if you take this on	13
7	Timing & formula reference	14
8	Modification cookbook	14

1 Two MAC implementations, one idea

INET ships two separate, unrelated C++ implementations of CSMA/CA-family MAC protocols. Knowing which one you’re looking at matters, because they don’t share code:

Module	Location	What it is
CsmaCaMac	src/inet/link-layer/csmaca/	A small, self-contained, generic CSMA/CA MAC with optional ACK+retry. Not 802.11-specific — the NED doc calls it “an imaginary CSMA/CA-based MAC protocol”. With the right parameters it approximates 802.11b ad-hoc behavior. One file, one state machine — the best place to first understand CSMA/CA mechanics before tackling the 802.11 stack.
Ieee80211Mac	src/inet/link-layer/- ieee80211/mac/	The real, standards-tracking 802.11 MAC: DCF, EDCA/HCF, RTS/CTS, fragmentation, block ack, rate control, NAV — heavily modularized into ~40 collaborating classes across a dozen subfolders. This is what <code>Ieee80211Interface</code> / <code>WirelessHost</code> actually use.

Section 2 covers `CsmaCaMac`. Sections 3–6 cover the 802.11 stack. Section 7 onward is reference material and a practical guide to making changes in either one.

2 Generic CSMA/CA: CsmaCaMac

SRC/INET/LINKLAYER/CSMACA/

`CsmaCaMac` (`CsmaCaMac.h` / `.cc`) implements the entire protocol in a single cFSM driven by the `FSMA_*` macro family (`inet/common/FSMA.h`). Every parameter, state and transition below is exactly what’s in the code — this is the cleanest reference implementation of “CSMA/CA + ACK + binary exponential backoff” in the framework.

2.1 Parameters (`CsmaCaMac.ned`)

Parameter	Default	Meaning
<code>useAck</code>	<code>true</code>	If false, every transmission is assumed successful — no retry, no ACK wait.
<code>bitrate</code>	—	Used for both data and ACK frames.
<code>headerLength</code>	/ 17B / 14B	Fixed-size MAC header / ACK frame.
<code>ackLength</code>		
<code>slotTime</code>	20 μ s	Backoff slot duration.
<code>sifsTime</code>	10 μ s	Gap before an ACK.
<code>difsTime</code>	<code>sifsTime</code> + 2· <code>slotTime</code>	Gap before contending for a new (non-retry) transmission — note this is computed from the other two, exactly like real DIFS.
<code>ackTimeout</code>	<code>ackLength</code> ·8/ <code>bitrate</code> + <code>sifsTime</code> + <code>slotTime</code>	How long the originator waits for an ACK before declaring failure.

Parameter	Default	Meaning
<code>cwMin / cwMax</code>	31 / 1023	Contention window bounds, in slots.
<code>cwMulticast</code>	= <code>cwMin</code>	Multicast/broadcast frames use a fixed window (no ACK, so no growth).
<code>retryLimit</code>	7	Max retransmissions before the frame is dropped.

2.2 The state machine

The FSM lives entirely in `CsmaCaMac::handleWithFsm()`. Eight states:

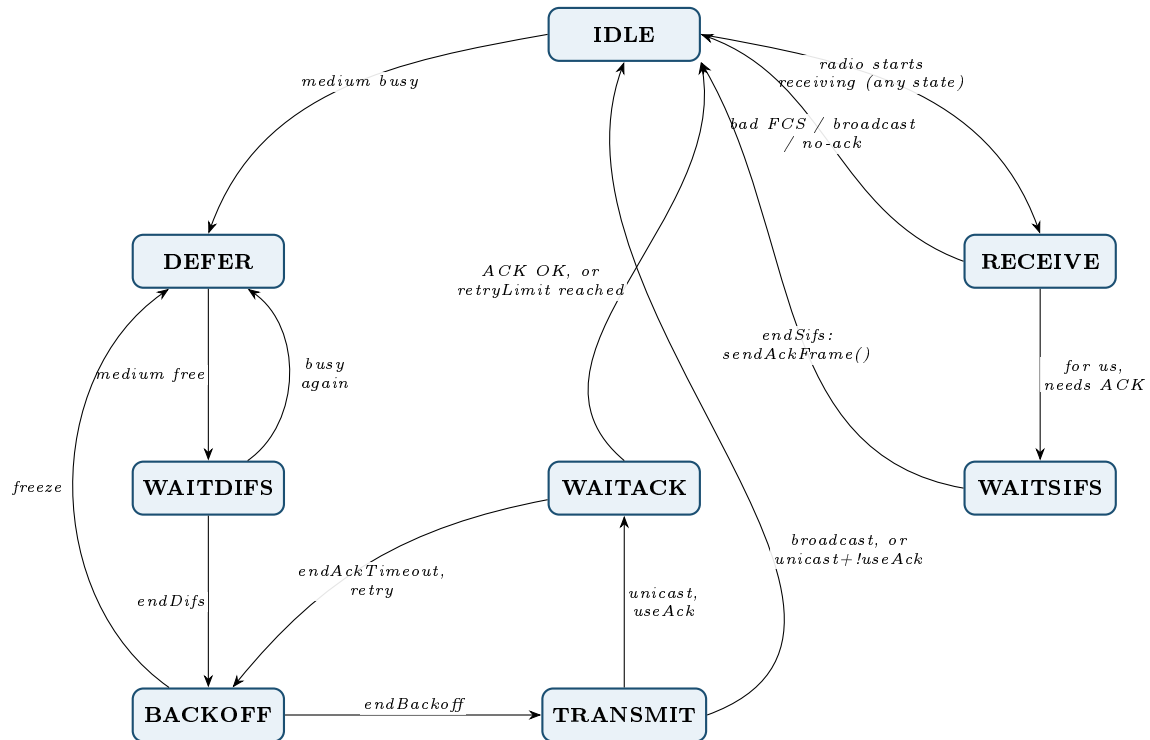


Figure 1: Simplified transition map of `CsmaCaMac`’s cFSM (see `handleWithFsm()`). The “radio starts receiving” edge fires from IDLE/DEFER/WAITDIFS/BACKOFF alike, which is how the MAC preempts its own backoff to receive.

State	Meaning
IDLE	Nothing pending. On entry back to IDLE, the FSM immediately re-checks: receiving? deliver to RECEIVE. Frame queued? re-enter the FSM with it. Queue non-empty? dequeue. This re-entrant check at the bottom of <code>handleWithFsm()</code> is what makes the MAC “self-driving” without extra self-messages for the common case.
DEFER	Something to send, but medium busy. Waits for <code>mediumStateChange</code> .
WAITDIFS	Medium just went free for a fresh (non-backed-off) frame; waiting out <code>difsTime</code> before starting backoff. Cancelled if the medium goes busy again or a reception starts.

State	Meaning
BACKOFF	Counting down the random backoff slots. If the medium goes busy mid-countdown, <code>decreaseBackoffPeriod()</code> freezes the remaining time (elapsed time is rounded down to whole slots) and the FSM returns to DEFER — the frozen counter resumes on next transition to WAITDIFS/BACKOFF, which is the textbook 802.11 “backoff survives across busy periods” rule.
TRANSMIT	Radio is transmitting the current frame (<code>sendDataFrame()</code>). On completion: broadcast or (unicast with <code>useAck=false</code>) → done, back to IDLE; unicast with ACK → WAITACK.
WAITACK	ACK timer running. Correct ACK for us → success, IDLE. Timeout at retry limit → give up (<code>giveUpCurrentTransmission()</code> , emits <code>linkBroken</code>). Timeout otherwise → <code>retryCurrentTransmission()</code> : increments <code>retryCounter</code> , regenerates the backoff period from the now-larger window, back to IDLE (which immediately re-drives into BACKOFF because <code>currentTxFrame</code> is still set).
RECEIVE	Reception in progress. On completion: bad FCS → count as collision, drop; unexpected ACK → ignore; broadcast or (unicast, no ACK policy) → deliver up, IDLE; unicast needing ACK → decapsulate+deliver up and move to WAITSIFS to send the ACK; not addressed to us → drop.
WAITSIFS	Waiting <code>sifsTime</code> before replying with an ACK to the frame just received (held via <code>endSifs</code> ’s context pointer). On expiry: <code>sendAckFrame()</code> , back to IDLE.

2.3 Backoff and contention window growth

The window is derived fresh every time from `retryCounter`, not stored and doubled incrementally (contrast this with the EDCA implementation in §5, which does the opposite). From `generateBackoffPeriod()`:

```

cw = min(cwMax, (cwMin + 1) * 2^retryCounter - 1) // binary exponential backoff
slots = uniform_random_int(0, cw) // intrand(cw + 1)
backoffPeriod = slots * slotTime

```

Multicast/broadcast frames always use `cwMulticast` regardless of `retryCounter` (they’re never acknowledged or retried, so there is no growth to speak of). `retryCounter` and the backoff state both reset to a clean slate in `resetTransmissionVariables()` once a frame is finished or given up — the next queued frame starts at `cwMin`, exactly as the standard specifies for DCF.

Where carrier sense comes from

`isMediumFree()` reads `radio->getReceptionState() == RECEPTION_STATE_IDLE` — physical carrier sense only. There is no NAV / virtual carrier sense in `CsmaCaMac` (no RTS/CTS, no duration field): that machinery is 802.11-specific and lives in the Rx/protection-mechanism classes covered in §4. If you want students to add virtual carrier sensing as an exercise, this module is the right size to do it in.

3 802.11 MAC architecture overview

SRC/INET/LINKLAYER/IEEE80211/MAC/

Unlike `CsmaCaMac`, the 802.11 MAC is deliberately split across many small, replaceable modules — the NED doc for `Ieee80211Mac` says so explicitly: “heavily modularized to enable customiza-

tion and extension of almost any aspect of its operation.” That is simultaneously why it’s harder to read on first pass, and why it’s the right codebase to modify for a research or course project: most changes are “write a new class implementing an existing interface and point a NED parameter at it,” not “edit a monolith.”

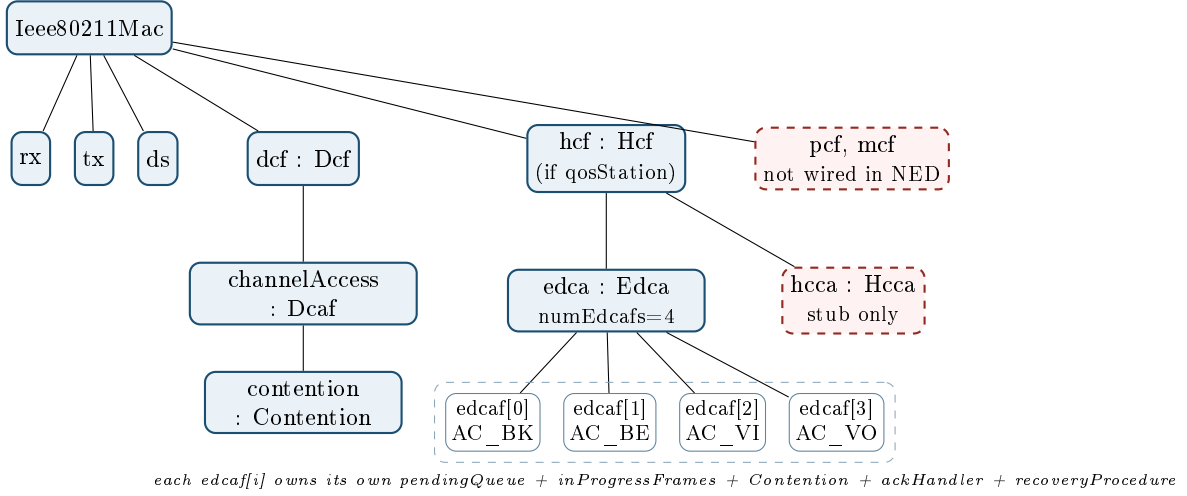


Figure 2: Submodule tree of a QoS-enabled `Ieee80211Mac`. Dashed red boxes exist as C++ classes and are referenced from `Ieee80211Mac.h`, but are either unimplemented stubs (`Hcca`) or never instantiated by the shipped NED file (`pcf, mcf`) — see §6.

3.1 Which class does what

Class	Role
<code>Ieee80211Mac</code>	The only module that’s a real <code>cSimpleModule</code> receiving/sending messages on its own behalf despite looking like a plain compound module. Owns radio-mode management, and statically dispatches every upper/lower frame to exactly one of <code>dcf</code> or <code>hcf</code> based on the <code>qosStation</code> parameter (<code>mib->qos</code>) — see <code>handleUpperPacket/handleLowerPacket</code> in <code>Ieee80211Mac.cc</code> . It is not a per-frame dynamic choice.
<code>Dcf</code>	DCF coordination function (§4). Wires together channel access, frame sequencing, RTS/CTS and ACK procedures, and recovery/retry for a single, AC-less queue.
<code>Hcf</code>	HCF coordination function (§5). In this codebase, effectively “EDCA” — it owns an <code>Edca</code> (4 access-category queues) and an <code>Hcca</code> that is a non-functional placeholder.
<code>Rx / Tx</code>	The only two modules that actually touch the radio. <code>Rx</code> tracks reception/-NAV state and tells every registered <code>IContention</code> instance (“registered” — plural, because EDCA has four) when the medium is free. <code>Tx</code> serializes transmit requests, applies the requested IFS gap, and reports completion back to whichever coordination function asked.
<code>Ds</code>	Distribution-service address translation (infra/AP addressing, ToD-S/FromDS bits) — outside the scope of this tutorial.

4 DCF (Distributed Coordination Function)

MAC/COORDINATIONFUNCTION/DCF · MAC/CHANNELACCESS/DCAF · MAC/CONTENTION/CONTENTION

4.1 Timing parameters

From `Dcaf::calculateTimingParameters()` (`Dcaf.cc`) — this is the actual code, not a paraphrase:

```
slotTime = modeSet->getSlotTime();
sifs = modeSet->getSifsTime();

// PIFS = aSIFSTime + aSlotTime (Eq. 9-2)
// DIFS = aSIFSTime + 2 x aSlotTime (Eq. 9-3)
ifs = (difsn == -1) ? sifs + 2*slotTime : difsn*slotTime;

// EIFS = aSIFSTime + DIFS + ACKTxTime(lowest mandatory rate) (Eq. 9-4)
eifs = sifs + ifs + modeSet->getSlowestMandatoryMode()->getDuration(LENGTH_ACK);

cwMin = (par cwMin == -1) ? modeSet->getCwMin() : par;
cwMax = (par cwMax == -1) ? modeSet->getCwMax() : par;
```

`sifs` and the PHY-derived `aCwmin/aCwmax` come from the active `Ieee80211ModeSet` (802.11a/b/g/n/p/ac have different SIFS and slot times — this is why `Ieee80211Mac` has a `modeSet` string parameter). EIFS is what a station falls back to after receiving a frame it couldn't decode (bad FCS): it can't rely on a NAV or duration field it never read, so it waits long enough that even the slowest possible ACK exchange would have finished.

4.2 The Contention module: backoff + IFS as one FSM

`Contention` (`mac/contention/Contention.cc`) is the shared backoff engine used by both DCF's `Dcaf` and every EDCA `Edcaf` access category — one instance per queue that can independently contend. It is a 3-state FSM (`IDLE` / `DEFER` / `IFS_AND_BACKOFF`), simpler than `CsmaCaMac`'s because it deliberately does not know about frame transmission — it only ever answers “when may I transmit,” via a callback (`ICallback::channelAccessGranted()`).

State	Entered when	Leaves on
IDLE	Nothing pending.	<code>startContention()</code> : medium free → straight to <code>IFS_AND_BACKOFF</code> ; medium busy → <code>DEFER</code> .
DEFER	Medium busy while something is pending.	Medium goes free → back to <code>IFS_AND_BACKOFF</code> (recomputing the wait). A corrupted frame received while deferring sets <code>endEifsTime</code> , so the next IFS uses EIFS instead of the normal IFS.
IFS_AND- _BACKOFF	Counting down <code>IFS + backoff-Slots*slotTime</code> via a single scheduled self-message (<code>startTxEvent</code>).	Timer fires → grants channel access (<code>IDLE</code>). Medium goes busy → back to <code>DEFER</code> , after <code>computeRemainingBackoffSlots()</code> freezes the countdown to whole elapsed slots (partial slots are not credited — you cannot save a fraction of a slot, matching the standard).

Backoff slot count is drawn once, in `startContention(cw, ...)`: `backoffSlots = intrand(cw`

+ 1) — the caller (`Dcaf/Edcaf`) is responsible for tracking and growing `cw` itself; `Contention` is stateless across calls.

Backoff optimization (on by default, `backoffOptimization=true`)

When contention starts from IDLE and the medium has already been free for a while, `scheduleTransmissionRequest()` subtracts the smaller of the already-elapsed free time and the already-elapsed idle time from the wait interval — i.e., it credits time the channel was already free just before the frame arrived, rather than restarting the whole DIFS+backoff wait from zero. This is a modeling nicety (real hardware can't know the future), not part of the standard; set the NED parameter to `false` for a stricter reading of 802.11.

4.3 Frame sequences: a small combinator grammar

This is the single most distinctive piece of architecture in INET's 802.11 MAC, and the main extension point for adding new frame exchanges (including, if you attempt it, PCF — see §6). Rather than hand-coding each possible frame exchange as imperative logic, `mac/frameSequence/GenericFrameSequences.h` defines four composable primitives that mirror the grammar notation Annex G of the 802.11 standard itself uses:

Combinator	Meaning
<code>SequentialFs</code>	Run element 0, then 1, then 2, ... in order (grammar's implicit concatenation).
<code>OptionalFs</code>	Run the wrapped element only if a predicate says so (grammar's <code>[...]</code>).
<code>RepeatingFs</code>	Re-run the wrapped element while a predicate holds (grammar's <code>{ ... }</code>).
<code>AlternativesFs</code>	Pick exactly one of several elements via a selector function (grammar's <code> </code>).

The leaves are primitive one- or two-frame exchanges in `PrimitiveFrameSequences.h`: `RtsFs`, `CtsFs`, `RtsCtsFs`, `DataFs`, `ManagementFs`, `AckFs`, `SelfCtsFs`, `FragFrameAckFs`, `LastFrameAckFs`, `BlockAckReqBlockAckFs`. `DcfFs` (`DcfFs.cc`) assembles these into DCF's actual grammar, quoted directly in the source from the standard's Annex G.2:

```
// frame-sequence =
// ( [ CTS ] ( Management + broadcast | Data + group ) ) |
// ( [ CTS | RTS CTS ] {frag-frame ACK } last-frame ACK )

AlternativesFs({
    SequentialFs({ Optional(SelfCts, isSelfCtsNeeded),
                  Alternatives({Management, Data}, selectMulticastDataOrMgmt) }},
    SequentialFs({ Optional(Alternatives({SelfCts, Sequential({Rts, Cts}})}, ...),
                  isCtsOrRtsCtsNeeded),
                  Repeating(FragFrameAck, hasMoreFragments),
                  LastFrameAck })
}, selectDcfSequence)
```

Concretely, for an ordinary unicast data frame with no fragmentation and RTS/CTS enabled, this evaluates to: `RTS → CTS → DATA → ACK`. Without RTS/CTS: `DATA → ACK`. For a broadcast frame: just `DATA` (no ACK, no protection). The `isRtsCtsNeeded` predicate defers to whatever `IRtsPolicy` is plugged in (§4.4); `isSelfCtsNeeded` is hard-coded false (self-CTS protection is not implemented for DCF).

`FrameSequenceHandler` (`FrameSequenceHandler.cc`) is the tiny driver that walks whatever `IFrameSequence` it's given: `prepareStep()` to get the next TRANSMIT or RECEIVE step, hand it to the coordination function's callback (`transmitFrame()` or `scheduleStartRxTimer()`), and on completion call `completeStep()` to advance. If a step is rejected (bad response, timeout) the whole sequence aborts and `originatorProcessFailedFrame/originatorProcessRtsProtectionFailed` fires on the coordination function, which is where retry/recovery logic hooks in (§4.5).

4.4 RTS/CTS, NAV and the duration field

`RtsPolicy` (`originator/RtsPolicy.cc`) decides per-frame whether to protect it: packet size $\geq$ `rtsThreshold` && !multicast. `CtsTimeout = SIFS + slotTime + PHY-RX-start-delay` by default (quoting the standard directly in the source comment). `OriginatorProtectionMechanism` (`protectionmechanism/OriginatorProtectionMechanism.cc`) computes the Duration/ID field that every originator-sent frame carries — this is what sets NAV (virtual carrier sense) on every station that overhears the frame, whether or not it's the intended recipient:

Frame	Duration/ID
RTS	CTS-time + pending-frame-time + ACK-time + 3·SIFS
Data/Mgmt, multicast	0 (no ACK expected, nothing to protect)
Data/Mgmt, unicast, last fragment	ACK-time + SIFS
Data/Mgmt, unicast, more fragments follow	next-fragment-time + 2·ACK-time + 3·SIFS

On the receive side, `Rx::setOrExtendNav()` (`Rx.cc`) reads this field out of every correctly-received frame (regardless of addressee) and extends its NAV timer if the new value would push it further into the future — NAV only ever grows monotonically from a single frame's perspective, exactly as virtual carrier sense requires. `Rx::isMediumFree()/recomputeMediumFree()` then combine physical carrier sense (from the radio) with this NAV state before telling any registered `Contention` instance that the medium is busy.

4.5 Retry, recovery, and contention-window growth

`NonQosRecoveryProcedure` is DCF's retry policy. It maintains a short retry count (for frames $\leq$ RTS threshold or already RTS-protected) and long retry count (for larger frames), against `dot11ShortRetryLimit/dot11LongRetryLimit`-style limits, and calls into `Dcaf::incrementCw()` / `resetCw()` on failure/success:

```
// Dcaf.cc
incrementCw(): cw = min(cwMax, 2*cw + 1); // doubles (roughly) on every failure
resetCw(): cw = cwMin; // back to minimum on success
```

Note the contrast with §2's `CsmaCaMac`: there, `cw` is recomputed from `retryCounter` each time $((cwMin+1) \cdot 2^n - 1)$; here, it's incrementally doubled $(2 \cdot cw + 1)$ and separately reset. Both converge on the same exponential-backoff shape, but if you're grading or comparing student modifications, know which formula you're looking at.

Frame lifecycle end states are driven from `Dcf::originatorProcessFailedFrame / originatorProcessRtsProtectionFailed` (`Dcf.cc`): retry limit not reached $\rightarrow$ set the frame's Retry bit and leave it in `InProgressFrames` for the next contention round; retry limit reached $\rightarrow$ drop it, emit `packetDropped` (reason `RETRY_LIMIT_REACHED`) and `linkBroken`.

4.6 A complete DCF transmission, end to end

1. `Dcf::processUpperFrame()` enqueues the packet in `Dcaf`'s `pendingQueue`, then calls `channelAccess->requestChannel(this)`.
2. `Dcaf::requestChannel()` starts Contention with the current `cw`, DIFS, EIFS, slot time.
3. Contention runs its 3-state FSM (§4); when the countdown expires it calls back `Dcaf::channelAccessGranted()`, which sets `owning=true` and calls `Dcf::channelGranted()`.
4. `Dcf::channelGranted()` builds a `FrameSequenceContext` and starts a fresh `DcfFs` in the `FrameSequenceHandler`.
5. The frame sequence (§4) decides, from the queued frame and the RTS policy, whether the first step is RTS or the data frame itself; `Dcf::transmitFrame()` computes rate, duration/NAV field (§4.4), and calls `Tx::transmitFrame()`.
6. At the recipient, if the frame needed protection, `CtsProcedure/ RecipientAckProcedure` fire the SIFS-gated CTS/ACK via `Dcf::transmitControlResponseFrame()`.
7. Each response is fed back through `Dcf::processLowerFrame()` → `frameSequenceHandler->processResponse()`, advancing the sequence, until it finishes (success) or a receive step times out (failure, §4.5).
8. `Dcf::frameSequenceFinished()` releases the channel and, if more frames are pending, immediately requests it again — restarting from step 2 with a freshly reset or grown `cw` depending on what just happened.



Figure 3: A protected DCF exchange as generated by `DcfFs`. Without RTS/CTS (frame below `rtsThreshold`), the sequence collapses to backoff → DATA → SIFS → ACK, and no NAV is set by the originator (only the duration field on DATA itself, per the table in §4.4). DIFS = SIFS + 2·slotTime (§4); the Duration/ID field on RTS & CTS equals the time remaining for CTS+DATA+ACK+3·SIFS.

5 HCF / EDCA (Enhanced Distributed Channel Access)

MAC/COORDINATIONFUNCTION/HCF · MAC/CHANNELACCESS/EDCA, EDCAF · MAC/CONTENTION/ED-CACOLLISIONCONTROLLER

Terminology check

The 802.11 standard defines HCF as the combination of two channel-access mechanisms available during different periods: EDCA (contention-based, prioritized, usable by any QoS station) and HCCA (polled, requires a Hybrid Coordinator/Access Point). In INET, the class named `Hcf` instantiates an `Edca` submodule and an `Hcca` submodule — but `Hcca` is an unimplemented stub (§6). In practice, “HCF” in this codebase means EDCA. Everything below is what actually runs.

5.1 Four independent contenders, one radio

Edca instantiates `numEdcafs` (default 4) `Edcaf` submodules, one per Access Category — `AC_BK` (background), `AC_BE` (best effort), `AC_VI` (video), `AC_VO` (voice), in that fixed index order (`mac/common/AccessCategory.h`). Each `Edcaf` is a nearly complete miniature DCF: its own `pendingQueue`, `inProgressFrames`, `Contention`, `QosAckHandler`, `QosRecoveryProcedure` and `TxopProcedure`. Frames from upper layers are classified into an AC by `Edca::classifyFrame()` and enqueued into that AC's queue only; each AC contends independently and simultaneously for the same physical medium.

5.2 Per-AC timing: AIFS instead of DIFS

From `Edcaf::calculateTimingParameters()`:

```
if SIFS + AIFSN[AC] * slotTime; // ifs used for this AC's Contention
aifs = SIFS + AIFSN[AC] * slotTime;
eifs = SIFS + aifs + ACKTxTime(slowest mandatory mode);
```

Default AIFSN (`Edcaf::getAifsNumber()`) and per-AC `cwMin/cwMax` derived from the PHY's DCF `aCWmin` (`Edcaf::getCwMin/getCwMax`):

Access Category	AIFSN	CWmin	CWmax	TXOP limit (OFD-M/HT)
AC_BK (background)	7	aCWmin	aCWmax	0 (no bursting)
AC_BE (best effort)	3	aCWmin	aCWmax	0
AC_VI (video)	2	$(aCWmin+1)/2 - 1$	aCWmin	3.008 ms
AC_VO (voice)	2	$(aCWmin+1)/4 - 1$	$(aCWmin+1)/2 - 1$	1.504 ms

These are exactly the 802.11e default EDCA parameter set. All four are overridable per-AC via the `aifsn/cwMin/cwMax` parameters on `edcaf[i]` in `omnetpp.ini` — no C++ change needed to retune priorities.

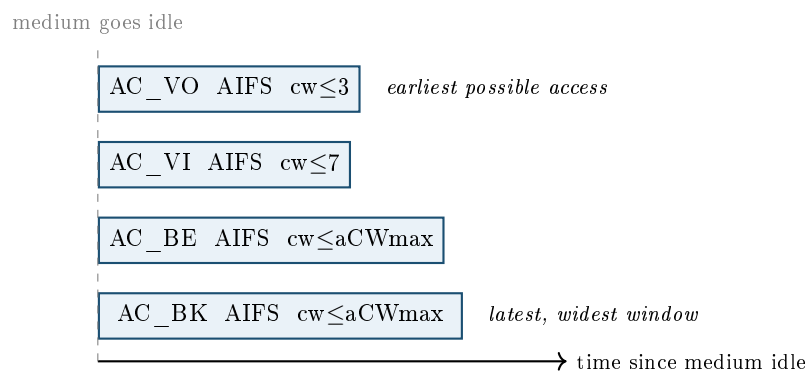


Figure 4: AIFS width and cw width both scale with `AIFSN[AC]`; bar widths above are illustrative, not to arbitrary scale. Why EDCA prioritizes traffic: a shorter AIFS lets `AC_VO` start counting down backoff slots sooner than `AC_BK` after the medium idles, and its narrower contention window makes a short backoff more likely — both effects stack in the same direction.

5.3 Internal (virtual) collisions

Because the four ACs contend independently inside the same station, two of them can legitimately finish backoff at the same simulated instant. This is an internal collision, resolved entirely in software with no channel involvement, via `EdcaCollisionController`:

```
// EdcaCollisionController.cc
isInternalCollision(edcaf):
    now = simTime();
    if txStartTimes[edcaf.ac] == now:
        for higherAc in (edcaf.ac+1 .. AC_VO): // higher index = higher priority
            if txStartTimes[higherAc] == now: return true;
    return false;
```

The asymmetry matters: `Edcaf::channelAccessGranted()` checks this before notifying its own coordination function — if it lost, it silently swallows its own grant and does nothing else. It's `Hcf::channelGranted()`, called on the winning AC, that discovers the losers (`edca->getInternallyCollidedEdcafs()`) and explicitly punishes each of them via `handleInternalCollision()`: their recovery procedure runs exactly as if an external collision had occurred (contention window doubles, retry counters increment, frame is dropped if the retry limit is now reached), and if they still have a frame to send, they immediately re-request the channel. The winning AC pays no penalty and proceeds straight into its frame sequence.

5.4 TXOP bursting

`TxopProcedure` gives `AC_VI` and `AC_VO` a non-zero TXOP limit (§5 table) — once a station wins the channel, it may send more than one frame back-to-back (SIFS-separated, no new backoff) until the TXOP limit is exhausted. `AC_BE` and `AC_BK` get a limit of zero, i.e. exactly one frame per channel win — same behavior as DCF. This is implemented as a separate frame-sequence grammar, `HcfFs / TxOpFs` (`framesequences/HcfFs.cc`, `TxOpFs.cc`), which — like `DcfFs` — is built from the same Sequential/Optional/Repeating/Alternatives combinators, quoting Annex G.3 directly:

```
// hcf-sequence =
// ( [CTS] 1{(Data+group[+QoS]) | Management+broadcast} +pifs ) |
// ( [CTS] 1{txop-sequence} ) <- unicast path, repeats while TXOP remains
```

`HcfFs::hasMoreTxOps()` is the loop condition: true for the very first frame of the TXOP, and thereafter true only while `txopProcedure->getRemaining() > 0` and the next queued frame isn't multicast. `SingleProtectionMechanism` computes the Duration/ID field for TXOP-bursting frames (the “single protection” variant of §4.4's calculation, extended to know about the TXOP's remaining time); “multiple protection” is defined by the standard but explicitly `throw cRuntimeError("Multiple protection is unsupported")` in `Hcf::transmitFrame()` if ever selected — it never is, because `TxopProcedure::selectProtectionMechanism()` is hard-coded to always return `SINGLE_PROTECTION`.

5.5 Block acknowledgement (optional)

If `Hcf`'s `isBlockAckSupported` parameter is true, two extra submodules are instantiated (`originatorBlockAckAgreementPolicy`, `recipientBlockAckAgreementPolicy`) and `Hcf` gains ADDBA/DELBA management-frame handling and a `BlockAckReqBlockAckFs` path through the TXOP grammar letting one `BlockAckReq/BlockAck` pair acknowledge many data frames at once. This tutorial doesn't dig into `mac/blockack/` in detail — treat it as the natural “next system to read” once DCF and plain EDCA are solid, using the same combinator/handler pattern

you already know from §4.

6 PCF and HCCA — status: not implemented

MAC/COORDINATIONFUNCTION/PCF · MAC/CHANNELACCESS/HCCA · MAC/FRAMESEQUENCE/PCFFS

Read this before assigning any PCF exercise

PCF and HCCA are not functional in this INET version. This isn't a case of “the feature is basic, extend it” — there is no working frame exchange, no polling logic, and no wiring from the top-level MAC module at all. `Ieee80211Mac.ned`'s own documentation states it plainly: “PCF is missing... HCCA is currently not implemented. (The module exists, but it is currently a placeholder.)” Below is exactly what does and doesn't exist, verified against the source, so a project to implement PCF starts from an accurate map rather than a surprise.

6.1 What exists today

File	Content	Status
<code>coordination-function/Pcf.h/.cc</code>	A class implementing <code>ICoordinationFunction</code> . Every method — <code>processUpperFrame</code> , <code>processLowerFrame</code> , <code>corruptedFrameReceived</code> — is <code>throw cRuntimeError("Unimplemented!")</code> .	STUB
<code>channelaccess/Hcca.h/.cc</code>	A class implementing <code>IChannelAccess</code> . <code>isOwning()</code> always returns <code>false</code> ; <code>requestChannel/releaseChannel</code> both throw. Every call site in <code>Hcf.cc</code> that reaches the “else if <code>hcca->isOwning()</code> ” branch does so only to immediately throw <code>cRuntimeError("Hcca is unimplemented!")</code> .	STUB
<code>framesequences/PcfFs.h/.cc</code>	An <code>AlternativesFs</code> with exactly two elements, both <code>new SequentialFs({})</code> — empty sequences. The source comment names what they're supposed to be ((<code>PS-Poll ACK</code>) and (<code>[Beacon+DTIM] {cf-sequence} [CF-End[+CFack]]</code>)) but neither is built. <code>selectPcfSequence()</code> always returns 0.	STUB
<code>Ieee80211Mac.ned</code>	Declares no <code>pcf</code> : submodule at all (only <code>dcf</code> , <code>hcf</code> if <code>QoS</code> , <code>ds</code> , <code>rx</code> , <code>tx</code>).	NOT WIRED
<code>Ieee80211Mac.h</code>	Still declares <code>opp_component_ptr<Pcf> pcf</code> ; and <code>opp_component_ptr<Mcf> mcf</code> ; member pointers — but since the NED file never instantiates a submodule named <code>pcf</code> or <code>mcf</code> , these pointers are simply never assigned and stay null. No code in <code>Ieee80211Mac.cc</code> reads them.	DEAD CODE

Net effect: even setting a station to “PCF mode” is not a supported configuration in the shipped NED — there is no parameter or submodule choice that instantiates `Pcf` at all. It is reachable only by writing new NED and C++.

6.2 What PCF/HCCA would need to do, conceptually

For context when designing an implementation exercise — this is standard theory, not a description of running INET code:

- **Superframe structure.** Time alternates between a Contention Period (CP, ordinary DCF/EDCA) and a Contention-Free Period (CFP). A Beacon announcing the CFP duration sets every station's NAV for the whole CFP, so DCF/EDCA stations defer for its entire length without needing to sense each individual poll.
- **PIFS access.** The Point Coordinator (PC, co-located with the AP) seizes the medium at the start of the CFP using PIFS (SIFS + 1 slot) — shorter than DIFS/AIFS, so it always wins against ordinary DCF/EDCA stations trying to start a new CP transmission at the same instant.
- **Polling loop.** The PC sends a sequence of CF-Poll frames (or Data+CF-Poll piggybacked frames) to stations on its polling list, one at a time, each followed by SIFS; a polled station with nothing to send responds with a null frame; the PC times out and moves to the next station if there's no response.
- **CFP termination.** A CF-End frame (or CF-End+CF-Ack) explicitly ends the CFP early and releases the NAV it had set, letting ordinary contention resume immediately rather than waiting for the announced duration to elapse.

6.3 A concrete starting point, if you take this on

Following the same architecture the rest of the MAC already uses (this is exactly why it's modularized this way):

1. **Add the missing frame types.** There is no PS-Poll, CF-Poll, or CF-End frame subclass alongside the existing ones in `Ieee80211Frame.msg` — check `Ieee80211MacHeader`'s subclasses first; you will likely need to add at least one.
2. **Write the primitive frame sequences.** `PrimitiveFrameSequences.h` has no `PollFs/CfEndFs` — you'd add classes shaped like the existing `RtsFs/CtsFs` (§4), each a small `IFrameSequence` with `startSequence/prepareStep/completeStep`.
3. **Fill in `PcfFs`'s two branches** using those new primitives plus the existing `Sequential/Optional/Repeating/Alternatives` combinators — the grammar comment already in the file is your specification.
4. **Implement `Hcca` for real:** it needs to track a polling list, know when the CFP starts/ends, and implement `IChannelAccess`'s `requestChannel/releaseChannel` using PIFS access instead of DIFS/AIFS+backoff (i.e., it largely bypasses `Contention` — a PC doesn't binary-exponential-backoff).
5. **Wire it into the NED and the dispatch logic.** Add a real `pcf`: submodule to `Ieee80211Mac.ned`, and — since `Ieee80211Mac` currently dispatches statically to exactly `dcf` XOR `hcf` based on `qosStation` (§3) — decide and implement how a station knows it's in a CFP right now and should route through `hcca` instead of `edca`. `Hcf::processLowerFrame()` and `channelGranted()` already have an `hcca` branch stubbed in with a throw — that's precisely where real logic needs to replace the exception.

This is a substantial project (realistically, several weeks for a small team), which is exactly why it's a good capstone-level exercise rather than a homework problem — and why it's worth knowing, before assigning it, that it is a “build it” exercise and not an “extend the existing thing” one.

7 Timing & formula reference

Quantity	Formula	Source
PIFS	$SIFS + 1 \cdot \text{slotTime}$	802.11 Eq. 9-2, quoted in <code>Dcaf.cc</code>
DIFS	$SIFS + 2 \cdot \text{slotTime}$	Eq. 9-3, <code>Dcaf.cc</code>
EIFS	$SIFS + DIFS + \text{ACKTx-Time}(\text{slowest mode})$	Eq. 9-4, <code>Dcaf.cc</code> / <code>Edcaf.cc</code>
AIFS[AC]	$SIFS + \text{AIFSN}[AC] \cdot \text{slotTime}$	<code>Edcaf::calculateTimingParameters()</code>
DCF backoff (<code>CsmaCaMac</code>)	$cw = \min(cwMax, (cwMin+1) \cdot 2^{\text{retry} - 1})$	<code>CsmaCaMac::generateBackoffPeriod()</code>
DCF/EDCA CW growth (802.11 MAC)	$cw \leftarrow \min(cwMax, 2 \cdot cw + 1)$ on failure; $cw \leftarrow cwMin$ on success	<code>Dcaf::incrementCw/resetCw</code> , <code>Edcaf</code> (same formula)
Backoff slots drawn	uniform integer in $[0, cw]$	<code>Contention::startContention()</code> , <code>CsmaCaMac::generateBackoffPeriod()</code>
RTS duration/NAV	$CTS + \text{Data} + \text{ACK} + 3 \cdot SIFS$	<code>OriginatorProtectionMechanism::computeRtsDurationField()</code>
Data duration/NAV (last fragment)	$ACK + SIFS$	<code>computeDataFrameDurationField()</code>
Data duration/NAV (more fragments)	$\text{nextFrag} + 2 \cdot \text{ACK} + 3 \cdot SIFS$	same
CTS timeout	$SIFS + \text{slotTime} + \text{PHY-RX-start-delay}$	<code>RtsPolicy::getCtsTimeout()</code>
RTS threshold policy	need RTS iff $\text{size} \geq \text{rtsThreshold}$ && unicast	<code>RtsPolicy::isRtsNeeded()</code>
TXOP limit, <code>AC_VI</code>	6.016 ms (DSSS/HR-DSSS) / 3.008 ms (OFDM/HT)	<code>TxopProcedure::getTxopLimit()</code>
TXOP limit, <code>AC_VO</code>	3.264 ms (DSSS/HR-DSSS) / 1.504 ms (OFDM/HT)	same
TXOP limit, <code>AC_BE</code> / <code>AC_BK</code>	0 (no bursting)	same

8 Modification cookbook

Concrete starting points for common assignments — the class you'd subclass or the file you'd edit, and what to watch for.

Goal	Touch	Notes
Change the back-off algorithm itself (e.g. try MILD, DIDD, or a non-binary growth rule)	<code>Contention::schedule-TransmissionRequest()</code> and whoever calls <code>startContention(cw, ...)</code> (<code>Dcaf/Edcaf::incrementCw</code>), or <code>CsmaCaMac::generateBackoff-Period()</code>	Decide up front which implementation you're changing (§2 vs §4/§5) — they compute <code>cw</code> completely differently (recomputed vs. incrementally doubled).
Add a 5th (custom) access category	<code>AccessCategory.h</code> enum, <code>Edca.ned</code> (<code>numEdcafs</code> , per-index defaults), <code>Edcaf::getAifs-Number/getCwMin/getCwMax</code> , <code>EdcaCollisionController</code> 's fixed-size-4 arrays	<code>EdcaCollisionController::tx-StartTimes</code> is hard-coded to 4 — this is the part most likely to silently break if you don't touch it.
Change RT-S/CTS trigger policy (e.g. adaptive threshold)	New class implementing <code>IRtsPolicy</code> , pointed to by the <code>rtsPolicy</code> submodule slot in <code>Dcf.ned/Hcf.ned</code>	Don't edit <code>RtsPolicy</code> in place unless you want to change the default for every user — implement the interface instead, the way the framework does for rate control.
Change ACK policy (e.g. no-ack for a class of traffic)	<code>IOriginatorAck-Policy/IRecipientAckPolicy</code> (non-QoS) or <code>IOriginatorQoS-AckPolicy/IRecipientQoS-AckPolicy</code> (QoS)	QoS ack policy also decides block-ack vs. normal-ack (§5) — check <code>computeAckPolicy()</code> .
Add a new frame exchange sequence entirely (e.g. attempt PCF, or a custom cooperative-relay exchange)	<code>GenericFrameSequences.h</code> combinators + new leaf class(es) in <code>PrimitiveFrameSequences</code> + a new top-level <code>*Fs</code> class + wire it in wherever <code>new DcfFs()/new HcfFs()</code> is currently constructed (<code>Dcf::channelGranted / Hcf::channelGranted</code>)	See §6.3 for a worked-through example.
Change internal-collision resolution (e.g. randomize instead of strict AC priority)	<code>EdcaCollisionController::is-InternalCollision()</code>	Remember the asymmetry in §5: the winner is never touched, only losers get punished — a “fair” redesign needs to handle winner bookkeeping too.
Change TXOP limits or which ACs get bursting	<code>TxopProcedure::getTxopLimit()</code> , or just set the <code>txopLimit</code> NED parameter per <code>edcaf[i]</code>	No C++ change needed for a per-scenario change — it's a NED parameter override of a computed default.
Add virtual carrier sense (NAV) to the generic <code>CsmaCaMac</code> Implement HCCA / PCF	<code>CsmaCaMac.h/.cc</code> : add a duration field to <code>CsmaCaMacHeader</code> , track a NAV timer alongside <code>isMediumFree()</code> See §6 in full	Good self-contained exercise before tackling the much larger 802.11 Rx/protection-mechanism machinery (§4.4). Capstone-scale, not homework.